LEUPHANA
UNIVERSITÄT LÜNEBURG

APPLIED MACHINE LEARNING FOR SMART AND CONNECTED SYSTEMS

05. A Practical Intro to Deep Learning and Recurrent Neural Networks

Project Updates

Teamname	Link github repo
Fanta 4	https://github.com/Patreeses/ML4SCS_Fanta4
Goonies	https://github.com/karstensjonah-design/ML4CS-group-goonies
Burk macht Bock	https://github.com/noahsa16/ML4SCS_Burk_macht_Bock
Gruppenraum 3	https://github.com/IsaKilic/ML4CS-group-gruppenraum3
TennisTracker	https://github.com/Marcel-vgl/ML4SCS
ChewML	https://github.com/karstensjonah-design/ML4CS-group-ChewML

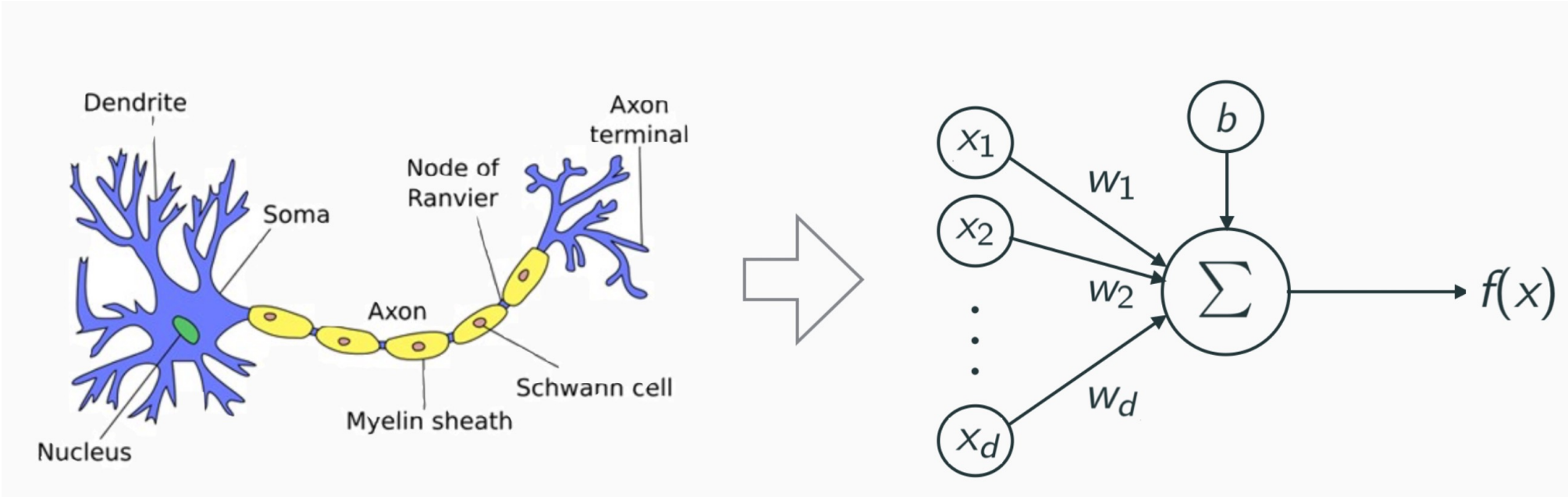
LEUPHANA
UNIVERSITÄT LÜNEBURG

FOUNDATIONS OF DEEP LEARNING

Multilayer Perceptrons & Optimization

Neurons/Perceptrons

- The perceptron was originally motivated by biology
- The model considers inputs x_i, synaptic weights w_i, and firing rates f

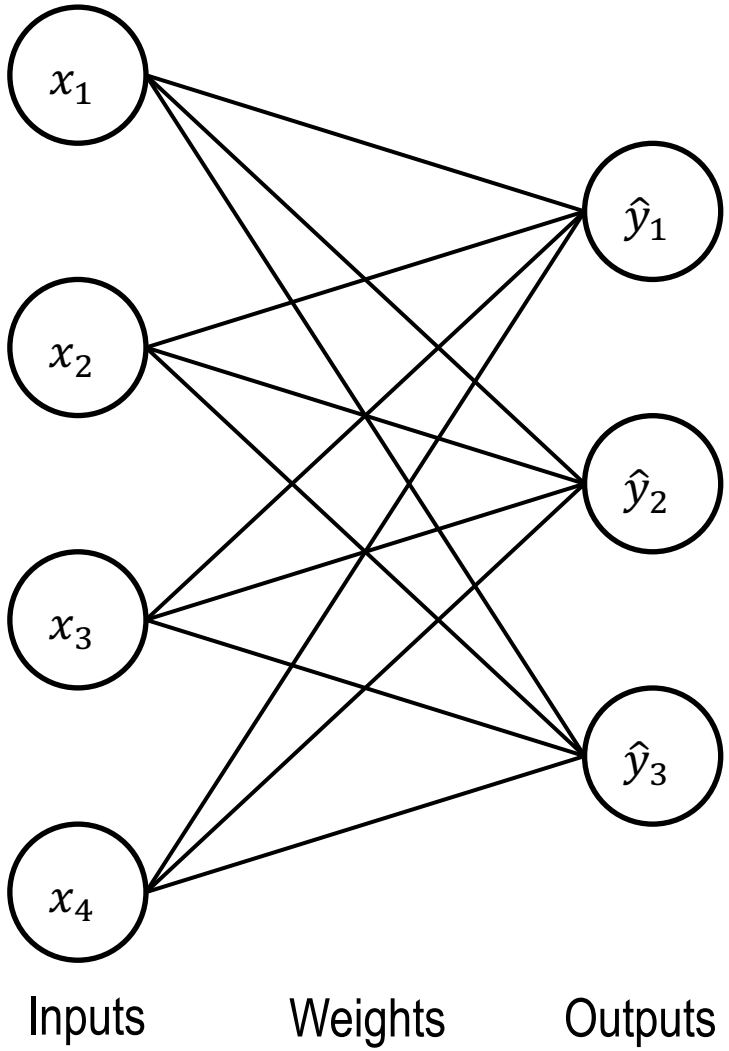


Layers

- We can stack multiple neurons to form a layer
- This yields multiple outputs $\hat{y}_i$ from the same inputs x_i as

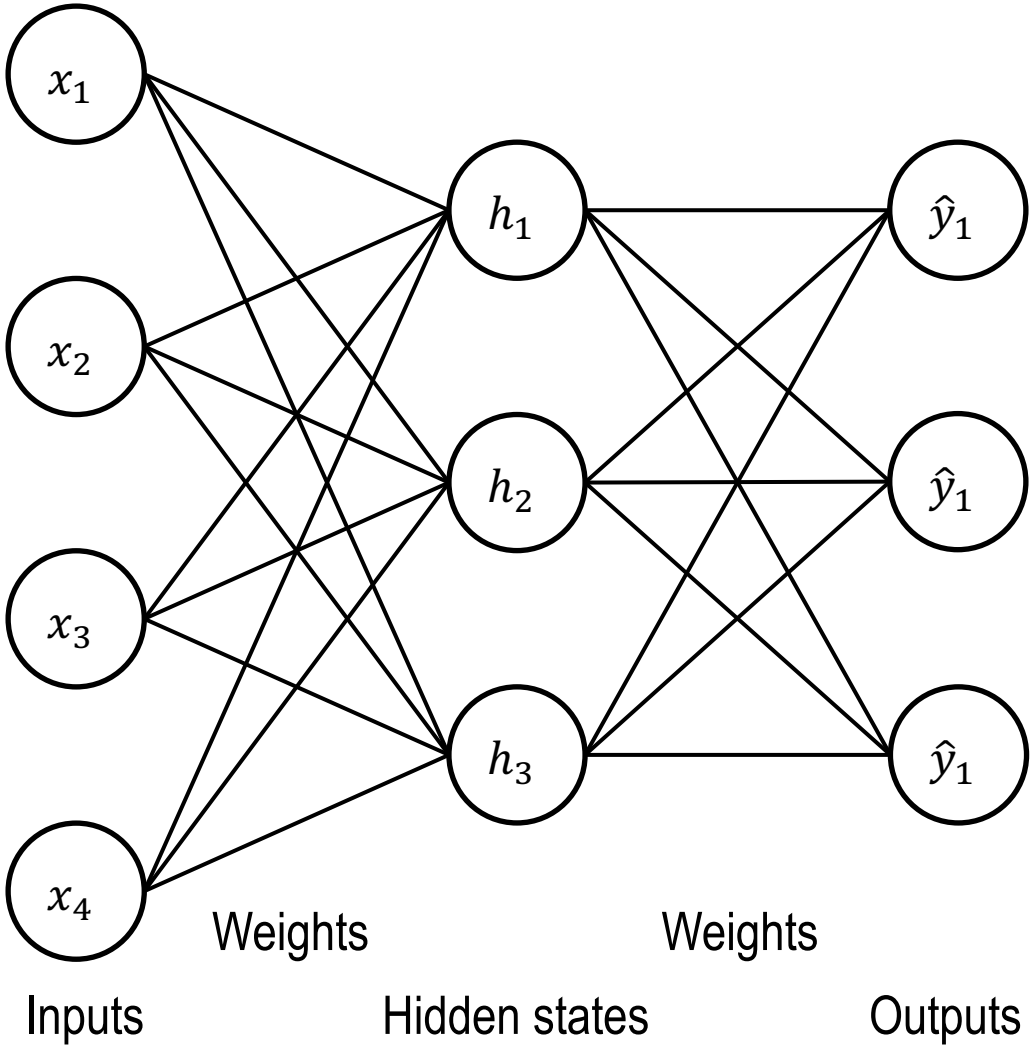
$$\hat{y}_i = f\left(\sum_j w_{ij} x_j\right)$$

- A use case of such a one-layer model could, e.g., be multi-label classification



Multilayer Perceptrons (MLP)

- We can chain multiple layers of perceptrons by using the output of one layer as an input for the next layer
- In this way, the input features get refined from layer to layer, allowing to realize complex mappings from input to output

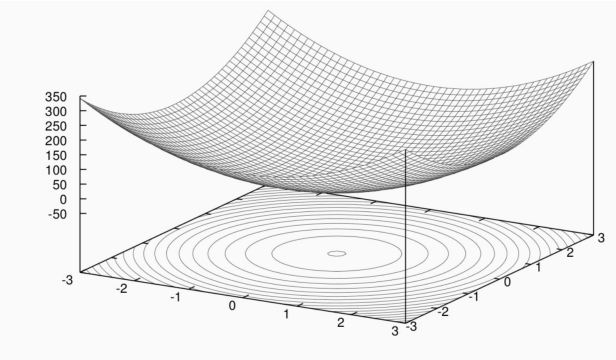


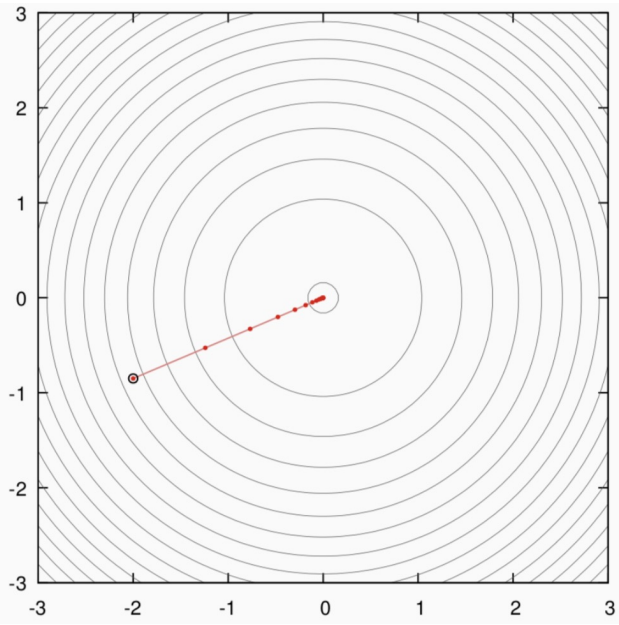
Universal Approximation Theorem (simplified)

A neural network *with one hidden layer and enough neurons is able to* ***approximate any given continuous function*** *defined on a finite domain* ***arbitrarily well****. In general, adding* ***more neurons*** *allows for* ***better approximations****.*

Optimization: Gradient Descend

- Gradients point in the direction of the steepest ascend
- Gradient descend uses this property to find a minimum of a function f:
  1. Compute the gradient at the current position w_t
  2. Update the position as $w_{t+1} = w_t - \nabla f(w_t)$
- In the context of neural networks, f is usually a loss function, and w the network weights





Optimization: Training Loop

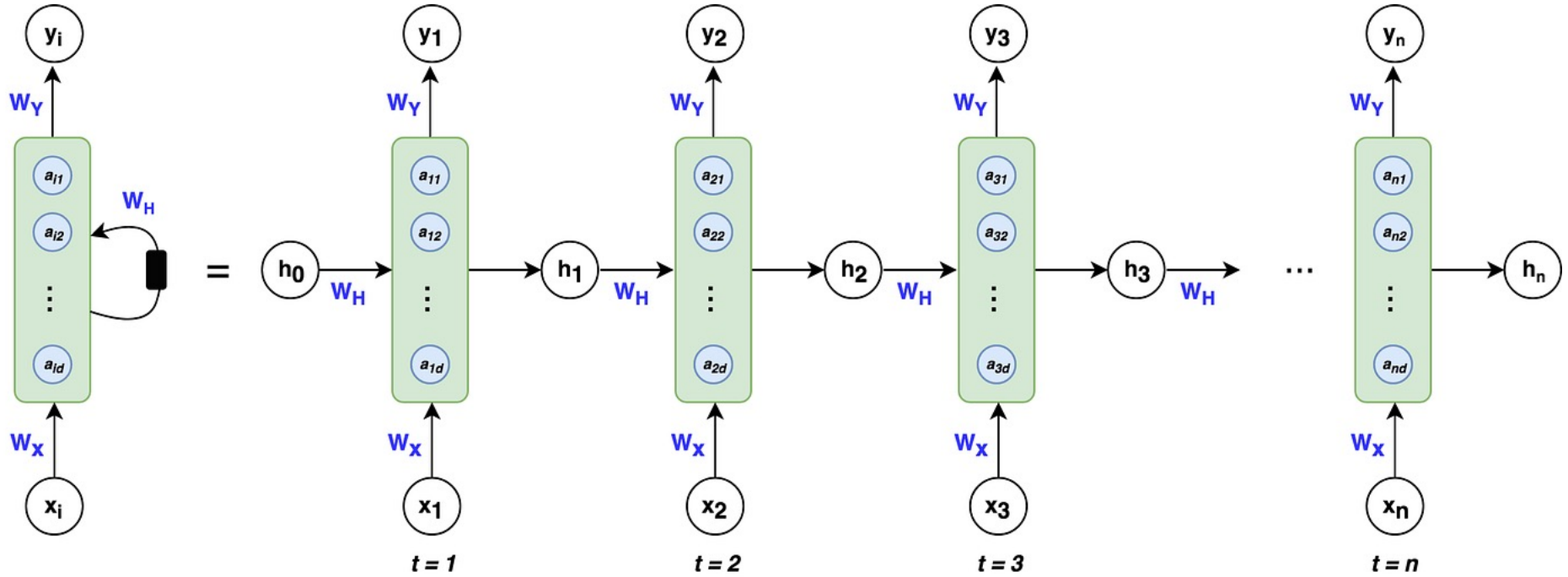
```
For each epoch:
      For each batch of training examples:
            Compute the output of the neural network
            Evaluate the loss function
            Compute the gradient  # [Backpropagation]
            Make a gradient step  # [Different optimizers available]
```

LEUPHANA
UNIVERSITÄT LÜNEBURG

RECURRENT NEURAL NETWORKS (RNN)

Concepts & Architectures

Basic RNN Architecture



Left: Shorthand notation often used for RNNs, **Right:** Unfolded notation for RNNs

Source (and further information): https://pub.towardsai.net/whirlwind-tour-of-rnns-a11effb7808f

Pros and Cons of RNNs

Advantages

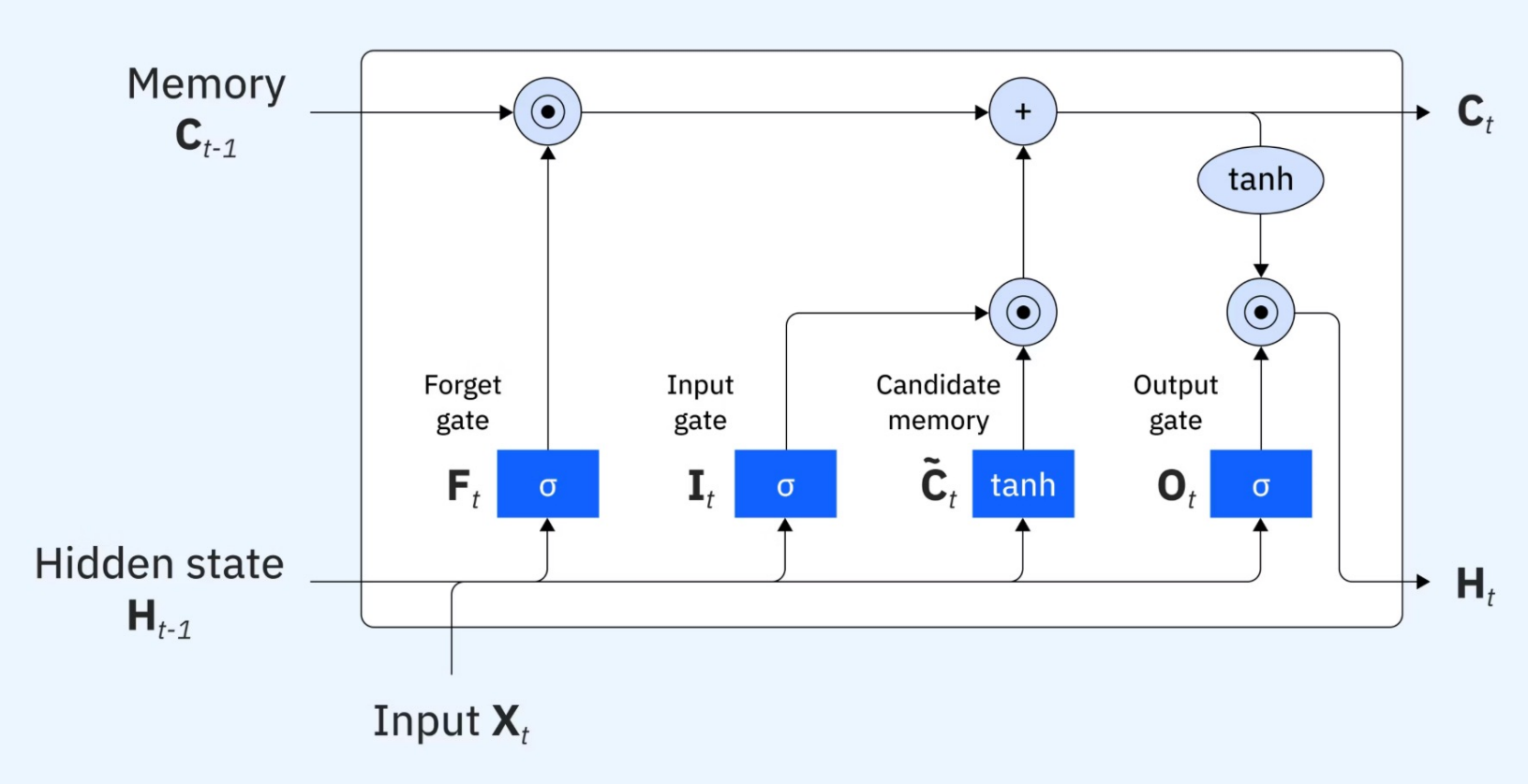
- Parameter efficiency: The same weights are used for all timesteps
- Directly addresses the order of sequential / time series data
- Can handle sequences of varying lengths

Disadvantages

- Information from early time steps can be „forgotten“, i.e., might not be retained in hidden states
- Backpropagation through time can cause vanishing/exploding gradients; optimization for long sequences can be tricky

Long Short-Term Memory (LSTM)

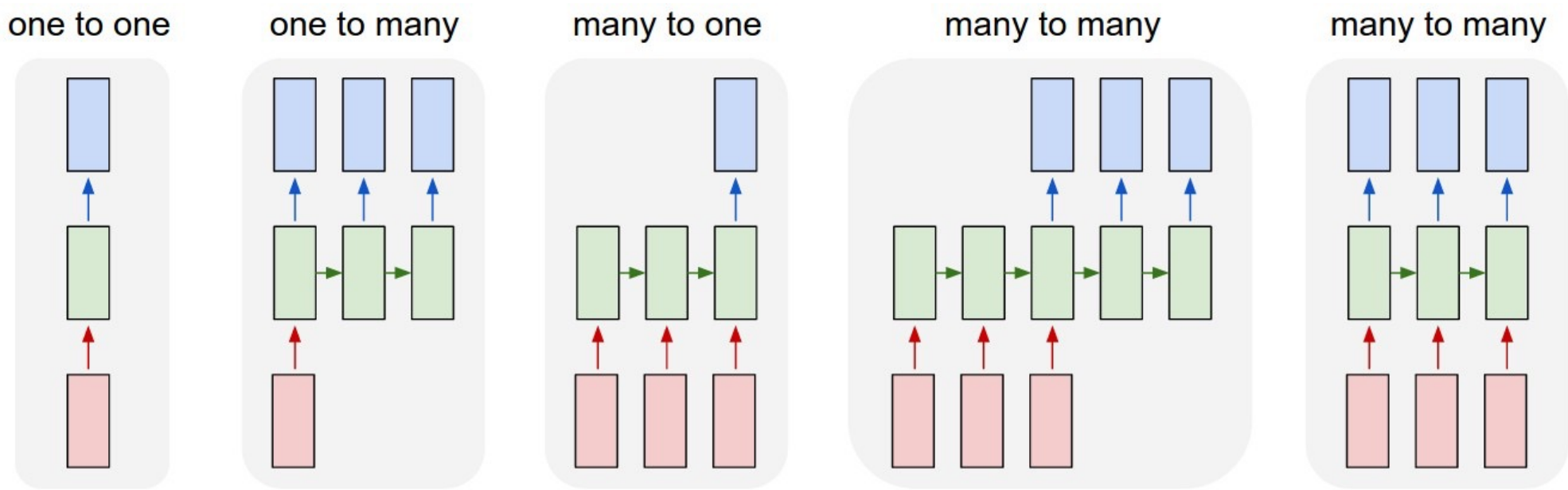
LSTMs mitigate forgetting and vanishing gradients by introducing a memory cell



Source: https://www.ibm.com/think/topics/lstm

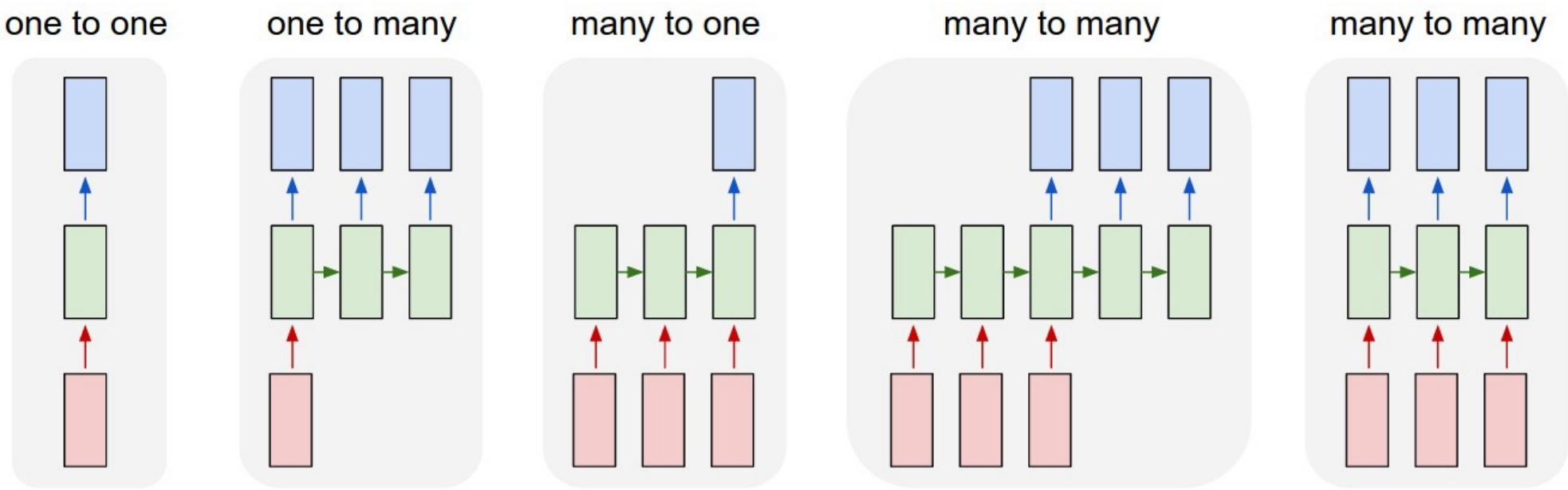
Types of RNN Architectures

RNNs can be used for diverse learning problems. **For which tasks would you employ the architectures depicted below?**



Source: http://karpathy.github.io/2015/05/21/rnn-effectiveness/

Types of RNN Architectures



Examples: (1) conventional neural network, e.g., for image classification, (2) natural language generation, (3) human activity recognition from sensor data (classification), (4) machine translation, (5) labelling individual frames of a video

Source: http://karpathy.github.io/2015/05/21/rnn-effectiveness/

LEUPHANA
UNIVERSITÄT LÜNEBURG

EXERCISE

RNN Implementation in PyTorch

Exercise - Car Engine Condition Classification Using LSTM

In this exercise, we will implement an LSTM-based classifier in PyTorch.

The dataset FordA contains sequences recorded with a sensor capturing engine noise, along with labels reflecting if the engine has a certain defect or not: https://www.timeseriesclassification.com/description.php?Dataset=FordA

Implement the classifier by resolving the #TODOs in the Jupyter Notebook: https://github.com/MaxHReinhardt/ML4SCS/blob/main/lstm_exercise.ipynb